

Hierarchical Expert Networks

A Tree-Structured Classification Framework with Modular, Extensible, and Sparse-Activated Components

TEAM Hengchun Song · Farhat Jahan · Divya Nambur Govindareddy

1 Project Description

When we look at the world, things naturally group together before they split apart: a tiger and a house cat are both cats before they are different species. But most neural network classifiers ignore this—they throw all classes into a flat list and treat a tiger–cat mix-up the same as a tiger–bus mix-up. This project builds a smarter alternative: a **Hierarchical Expert Network (HEN)** that mirrors how the world is actually organized. The network first asks a simple question—*“is this a digit, a letter, or a Chinese character?”*—then hands the input off to a specialist sub-network that only deals with that one category. The result is a system that matches the accuracy of a conventional classifier, but is easier to extend with new languages, cheaper to fix when one part breaks, faster at inference because two-thirds of the network stays idle per input, and faster to train because each specialist can be trained on a separate machine at the same time.

2 Background & Motivation

The World Is Not a Flat List

When a child learns to recognize animals, they do not memorize a flat list of species. Instead, they build a mental hierarchy: first learning that some animals are “cats” and others are “dogs,” then gradually distinguishing lions from tigers. This layered understanding reflects something fundamental about how categories exist in the real world—**objects cluster into nested groups**. Modern neural networks largely ignore this structure, penalizing a tiger–cat confusion equally to a tiger–bus confusion, even though the former is a far more understandable error.

The structure of errors matters. And the structure of the world—hierarchical, nested, grouped—should be reflected in how we build classifiers.

Flat Labels Are a Modeling Lie

In this project, a flat model treats all 34 classes as equally distant from one another. But in reality, the label space has a natural tree structure:

📁 All Classes (34 total)

├ **Digits** 10 classes • 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

├ **Letters** 9 classes • A, B, C, D, E, F, G, H, J

├ **Chinese** 15 classes • 一, 二, 三, 四, 五 ...

├ **Hindi** — to be added in future work

└ **Other ...** — extensible without retraining

3 Datasets

Three publicly available handwriting datasets are combined into a unified 34-class corpus. All images are resized to **32×32 grayscale** before training.

Digits

MNIST

70,000 handwritten digit images (0–9)

60k train / 10k test

28×28 grayscale

yann.lecun.com/exdb/mnist

Letters

EMNIST Letters

56,601 images, 9 uppercase letters

(A–J, excluding I)

28×28 grayscale

nist.gov — EMNIST Dataset

Chinese

Chinese MNIST

15,000 images, 15 Chinese numeral characters

80 / 20 stratified split

64×64 → resized to 32×32

kaggle.com/datasets/gpreda/chinese-mnist

4 Deep Learning Algorithms & Techniques

Four model variants are implemented and compared, along with the following techniques:

Hierarchical CNN
3-block Conv backbone + coarse router + 3 fine-grained expert heads

Flat CNN Baseline
Single monolithic CNN, 34-class direct output — comparison reference

Hierarchical MLP
Shared FC backbone + hard-routed expert heads — demonstrates sparse activation

Flat MLP Baseline
2-hidden-layer MLP, 34-class output — MLP comparison reference

Two-Phase Training
Phase 1: train backbone + router. Phase 2: freeze backbone, train experts independently

Hard MoE Routing
Router selects exactly one expert per input — 29% parameters dormant at inference

AdamW + Cosine LR
AdamW optimizer with cosine annealing learning rate schedule

Batch Normalization
Applied after every linear/conv layer to stabilize training

Dropout (p = 0.5)
Regularization in fully-connected layers to prevent overfitting

Class-Weighted Loss
Inverse-frequency weights in cross-entropy to handle class imbalance

Masked Cross-Entropy
Fine-grained loss computed only on samples belonging to each expert's domain

Confusion Matrix Analysis
Global (34×34), coarse (3×3), and per-domain matrices; within- vs cross-group error breakdown

5 **Project Timeline & Milestones**

 Wk 1–2

Environment Setup & Data Preparation ✓ Done

 Wk 3–4

Flat Baselines ✓ Done

Wk 5–6

Hierarchical Models In Progress

Wk 7

Evaluation & Analysis

Wk 8

Report & Presentation

6 Task Allocation

Member	Role
Hengchun Song	Architecture
Farhat Jahan	Data & Pipeline
Divya Nambur Govindareddy	Evaluation